OTP2/AD (2026)
Group 3
Sprint 6: Database /Localization

Database Localization Design - Implementation Report

1. Introduction

The goal of this localization design is to enable Fliply to support multiple languages for both static UI text and dynamic content stored in the database. The system must allow new languages to be added without modifying source code, and translations must be manageable directly through the database.

To achieve this, the solution introduces:

- A **Language** table to store all supported languages.
- A **LocalizationString** table to store all static UI text translations.
- An **Excel-based import mechanism** to populate static translations.
- Additional **translation tables** for dynamic entities (e.g., Class, FlashcardSet, Flashcard).

This approach ensures scalability, maintainability, and flexibility for future language expansion.

2. Current Database Overview

The existing database includes entities such as:

- **User**
- **Class**
- **FlashcardSet**
- **Flashcard**
- **Quiz**
- **Study**
- **Class_Details**
- **Quiz_Details**

These tables currently store text in a single language and do not support multilingual content.

3. Localization Architecture Overview

The proposed localization system consists of three major components:

3.1 Language Table

Stores all languages supported by the system.
New languages can be added simply by inserting a new row.

3.2 LocalizationString Table

Stores translations for **static UI text** such as:

- Buttons
- Labels
- Error messages
- Menu items
- System messages

Data is imported from an Excel file located in the project's resources folder.

3.3 Dynamic Translation Tables

For entities whose content varies (e.g., class names, flashcard terms), dedicated translation tables are created:

- FlashcardSet_Translation
- Flashcard_Translation
- User_Translation

4. Language Table Design

4.1 Table: Language

```
CREATE TABLE Language (
                        id          BIGINT PRIMARY KEY,
                        code        VARCHAR(10) NOT NULL UNIQUE,   -- e.g., 'en', 'vi', 'fi'
                        name        VARCHAR(100) NOT NULL,         -- English, Vietnamese, Finnish
                        is_default  BOOLEAN NOT NULL DEFAULT FALSE
);
```

Purpose

- code is used in the language menu and is also used as a reference key by other tables when querying data in a specific language.
- is_default defines fallback behaviour when a translation is missing.

5. Static UI Text Localization

5.1 Excel-Based Translation Source

A file such as: src/main/resources/localization/localization.xlsx

contains:

Key	EN	FI	KO	LO	VI
language.current	Language	Kieli	언어	ພາສາ	Ngôn ngữ
welcome.subtitle	Fliply	Fliply	Fliply	Fliply	Fliply
welcome.subtitle	Quick and Easy Learning	Nopea ja helppo oppiminen	빠르고 쉬운 학습	ການຮຽນຮູ້ງ່າຍ ແລະໄວ	Học nhanh và dễ dàng
welcome.register	Register	Rekisteröidy	회원가입	ລົງທະບຽນ	Đăng ký
welcome.login	Login	Kirjaudu sisään	로그인	ເຂົ້າສູ່ລະບົບ	Đăng nhập

5.2 Table: LocalizationString

```sql
CREATE TABLE LocalizationString (
                                id           BIGINT PRIMARY KEY,
                                key          VARCHAR(255) NOT NULL,
                                language_id  BIGINT NOT NULL,
                                text         TEXT NOT NULL,
                                CONSTRAINT fk_localization_language
                                    FOREIGN KEY (language_id) REFERENCES Language(id),
                                CONSTRAINT uq_localization_key_lang
                                    UNIQUE (key, language_id)
);
```

5.3 ImportService (Java)

A Java service reads the Excel file and inserts/updates translations.

Pseudo-code:

```java
for (Row row : sheet) {
    String key = getCell(row, "key");
    for (Language lang : languagesFromDb) {
        String value = getCell(row, lang.getCode());
        if (value != null && !value.isBlank()) {
            upsertLocalizationString(key, lang.getId(), value);
        }
    }
}
```

This allows non-developers (e.g., translators) to update UI text without modifying code.

6. Dynamic Content Localization

Static UI text is not enough — many entities contain user-facing text that must also be translated.

Example: Class Translation

Original table:

```sql
CREATE TABLE `CLASS`
(
    ClassId   INT NOT NULL AUTO_INCREMENT,
    ClassName VARCHAR(100) NOT NULL,
    TeacherId INT NOT NULL,
    PRIMARY KEY (ClassId),
    FOREIGN KEY (TeacherId) REFERENCES `USER`(UserId)
);
```

Translation table:

```sql
CREATE TABLE Class_Translation
(
    Id           INT NOT NULL AUTO_INCREMENT,
    ClassId      INT NOT NULL,
    LanguageId   INT NOT NULL,
    Name         VARCHAR(100) NOT NULL,

    PRIMARY KEY (Id),

    CONSTRAINT fk_class_translation_class
        FOREIGN KEY (ClassId) REFERENCES CLASS(ClassId)
        ON DELETE CASCADE,

    CONSTRAINT fk_class_translation_language
        FOREIGN KEY (LanguageId) REFERENCES Language(Id),

    CONSTRAINT uq_class_lang
        UNIQUE (ClassId, LanguageId)
);
```

7. Runtime Localization Behaviour

7.1 Language Selection

At runtime, the system must identify which language should be used to display content. Language selection occurs entirely within the application logic. The system determines the active language through the following mechanisms:

a. User Profile Language

Each user has a preferred language stored in the USER table (e.g., Language = 'en').
When the user logs into the system, this value becomes the default language for all localization operations.

b. User Interface Language Menu

The application provides a language selection menu that allows users to switch languages while using it manually.
When a user selects a language from the menu, the selected language code (e.g., "vi", "fi") overrides the profile language for the current session.

c. Default System Language

If a translation is not available in the selected language, the system falls back to the default language defined in the Language table (is_default = TRUE).
This ensures that the application always displays meaningful text, even when certain translations are missing.

7.2 Static Text Localization

Static UI text refers to all fixed interface elements, such as button labels, menu items, system messages, and error messages. These strings do not depend on user-generated content and are stored in the LocalizationString table.

Retrieval Process

1. The application identifies the text key associated with the UI element (e.g., "button.save").
2. The system determines the active language code based on Section 7.1.
3. A database query retrieves the corresponding translation.

SQL Query for Static Text

```
SELECT ls.text FROM LocalizationString ls
JOIN Language l ON l.id = ls.language_id
WHERE ls.key = :key AND l.code = :lang;
```

Fallback Behavior

If no translation exists for the selected language, the system retrieves the default language version:

```
... OR l.is_default = TRUE
```

This fallback mechanism guarantees that the UI remains functional and readable even when translations are incomplete.

7.3 Dynamic Content Localization

Dynamic content refers to user-generated or database-driven text, such as class names, flashcard terms, flashcard definitions, and flashcard set subjects.
Because these values vary across records, they require dedicated translation tables (e.g., Class_Translation, Flashcard_Translation, FlashcardSet_Translation).

Retrieval Process

1. The system loads the base record from the primary table (e.g., CLASS).
2. It attempts to retrieve a translated version from the corresponding translation table using the active language.
3. If a translation exists, it is used; otherwise, the system falls back to the default value stored in the base table.

SQL Query Example: Class Name Localization

```
SELECT  c.ClassId, COALESCE(ct.name, c.ClassName) AS ClassName
FROM CLASS c
LEFT JOIN Class_Translation ct
   ON ct.class_id = c.ClassId
```

```
AND ct.language_id = :langId;
```

Explanation

- ct.name contains the translated class name.
- c.ClassName is the default (usually English) value.
- COALESCE(ct.name, c.ClassName) ensures that the system always returns a valid name, even if the translation is missing.

8. Advantages & Trade-offs

Advantages

- New languages can be added without code changes.
- Static and dynamic content are both fully translatable.
- Excel import allows non-developers to manage translations.
- Scalable for large systems.

Trade-offs

- More database joins required.
- Slightly more complex schema.
- Requires caching for high-performance systems.